

MIDTERM (SHORT NOTES)

2024 - 2025

CS201 Important Key points

Contact Us: 03254755408

Lecture 1 – What Is Programming?

A program is a precise sequence of steps to solve a problem.

Programming builds critical reading, analytical thinking, and creative synthesis.

Essential skills: attention to detail, commenting, designing for reusability & UI, understanding that computers follow instructions literally.

Follow a design recipe: analyze the problem, abstract with examples, write precise specs, test and refine.

Lecture 2 – Software & Development Environment

Software categories: system (OS, drivers, utilities) vs. application software.

History of C: evolved from BCPL and B, standardized by ANSI/ISO in 1989.

Development tools: text editor → preprocessor → compiler → linker → loader → debugger.

Utilities: compression, defragmentation—manage disk performance.

Lecture 3 – First C Programs & Basics

Structure: `#include <iostream.h>, main(), { }, statements ending in ;.`

I/O streams: `cout <<` for output, `cin >>` for input.

Variables & data types: `int, short, long, float, double, char.`

Assignment = vs. comparison ==; arithmetic operators `+` `-` `*` `/` `%`; operator precedence and use of parentheses.

Lecture 4 – Expressions & Sample Programs

Build programs step by step: declare variables, prompt input, compute, display.

Integer division truncates decimals; use floats for real-number results.

Use clear prompts (`cout`) and `cin` for robust user interaction.

Practice writing and tracing expressions, enforce evaluation order with parentheses.

Lecture 5 – Decision Making (if)

`if (condition)` executes its block when true; braces `{ }` group multiple statements.

Relational operators: `>` `<` `>=` `<=` `==` `!=`.

Logical operators introduced later; use flowcharts (diamond symbol) to design decision logic.

Lecture 6 – if/else & Logical Operators

Extend decisions with `else` and `else if` for two-way or multi-way branching.

Logical operators: AND `&&`, OR `||`, NOT `!`; combine multiple conditions.

Beware using single `=` instead of `==` in conditions—leads to logical bugs.

Lecture 7 – for Loops

`for (init; condition; update)` is ideal when the iteration count is known.

Order: initialization → test → body → update → repeat.

Common pattern: `for (int i = 0; i < N; i++) { ... }.`

Lecture 8 – while & do-while Loops

`while (cond)` checks before entry; may execute zero times.

`do { ... } while (cond);` checks after body—executes at least once.

Use `break` to exit loops early, `continue` to skip to the next iteration.

Lecture 9 – Functions & Modular Design

Divide code into reusable functions: declaration (signature) vs. definition (body).

All programs start with `main()`, which may call other functions.

Top-down design: break complex tasks into subtasks, implement and test each.

Lecture 10 – Header Files & Scope

`#include` pulls in declarations (e.g., `<iostream.h>`).

Scopes: global (file scope), function-level, block-level—minimize globals.

Parameter passing: default is **call by value** (copies), optional **call by reference** (addresses).

Lecture 11 – Arrays & Linear Search

Arrays: contiguous memory, zero-based indexing.

Declaration: `int a[10];`; initialization: `= {0}` or loop-based.

Copy arrays via loops; search linearly with `for (i = 0; i < n; i++)`
`if (a[i]==key) ....`

Use `const` to protect arrays from modification.

Lecture 12 – Character Arrays (Strings)

Strings are `char` arrays terminated by `'\0'`.

Print with `cout << name;` or loop through characters.

Compare arrays element-wise or use `strcmp` in C++; sort/search arrays of strings similarly.

Lecture 13 – Array Manipulation & Design Recipes

Apply design recipe to real-world problems: analyze, break into functions, use arrays for related data.

Manipulate multidimensional data (e.g., grids) with nested loops.

Emphasize planning with examples before coding.

Lecture 14 – Pointers Part 1

Pointers store addresses: `int *p;` declares a pointer to `int`.

Address-of `&`, dereference `*`; null pointers initialized to `NULL`.

Pointer arithmetic moves by `sizeof(type)`; comparisons valid within the same array.

Lecture 15 – Pointers & Arrays

Array name decays to a pointer to its first element.

`arr[i] ≡ *(arr + i);` pointer arithmetic and indexing interchangeably.

Passing arrays to functions by pointer plus size; pointer comparisons only within one array.

Lecture 16 – Multidimensional Arrays & Pointers Continued

Declare 2D arrays: `int m[rows][cols];`; access with `m[i][j]`.

Pass 2D arrays as `func(int m[][COLS], int rows)`.

Pointer concepts extend to higher dimensions; nested loops for traversal.

Lecture 17 – String-Handling Functions

C `<cstring>` functions: `strlen`, `strcpy`, `strcat`, `strcmp`, `strstr`.

Conversion: `atoi`, `atof`, `strtol`.

Beware of buffer overflows—use `strncpy`, `strncat`, `strncmp` for safety.

Lecture 18 – Text File I/O

Use `<fstream>`: `ifstream` for input, `ofstream` for output, `fstream` for both.

Open modes: `ios::in`, `ios::out`, `ios::app`, `ios::binary`; check with `if (!fs)`.

Read with `getline` or `>>`, write with `<<`; close with `close()`.

Lecture 19 – Random-Access Files

Sequential vs. random access; use `seekg/tellg` (input) and `seekp/tellp` (output).

Efficient I/O: `read(buffer, size)` and `write(buffer, size)` for block transfers.

Insert or reverse-copy data by seeking to positions and reading/writing accordingly.

Lecture 20 – Structures & Unions

`struct Name { ... };` packs related fields; end with `;`.

Access with `.` for objects, `->` for pointers.

`sizeof` reports total size (including padding).

`union` shares memory among members—useful for variant data types.

Lecture 21 – Bit Manipulation

Bitwise operators: AND `&`, OR `|`, XOR `^`, NOT `~`; shift left `<<`, right `>>`.

Use masks and flags: set (`|=`), clear (`&=~`), toggle (`^=`), test (`&`).

Operate on `unsigned int` to avoid sign-extension issues.

Lecture 22 – Operator Review & Control Structures

Assignment variants: +=, -=, *=, /=, <<=, >>=, etc.

Nested if, switch with case, break; loops (while, do-while, for) with continue/break.

Reinforce design recipe steps and best practices: clear variable names, comments, consistent formatting.

LMS Handling Services

Get Expert Help with Your Learning Management System!

Half Semester | Full Semester | Single Activities

Contact Us: 03254755408

Virtual University Of Pakistan

MIDTERM

Exams Alert! +

Struggling to prepare for your midterm exams?

We've got the solution!

Comprehensive Study Material Available:

+ MCQ's File

+ Short Questions File

+ Updated Past Papers

+ Important Topics

Get instant access to our paid notes!

If you're interested,

Contact us: 03254755408

Thank you!